# Conversation ID in Spring AI ChatMemory

## 1. Introduction

Large Language Models are stateless by design. Every API call you make to a model is treated as a brand new conversation. The model has no memory of what was said before, unless you explicitly send the previous messages along with the new one.

This is where Spring AI ChatMemory comes in. It gives your Spring Boot application a clean way to store, manage, and replay conversation history with the model, so users feel like they are talking to something that actually remembers them.

At the heart of this system is one small but critical concept: the Conversation ID. This document explains what it is, why it matters, how to use it correctly, and what changed across Spring AI versions, with reference to the exact error students faced in class.

## 2. What is a Conversation ID

A Conversation ID is a unique string that identifies a single chat thread between a user and the AI model. Spring AI uses this ID as a key to store and retrieve all messages that belong to that specific conversation.

Think of it like a chat room ID. Two different users get two different rooms, and the model only sees the messages of the room you point it to.

### How Spring AI uses it internally

The default in-memory implementation stores chat history in a simple map:

```
public final class InMemoryChatMemoryRepository implements ChatMemoryRepository {
    Map<String, List<Message>> chatMemoryStore = new ConcurrentHashMap<>();

    // Conversation ID -> list of messages
}
```

When you send a new message, the **`MessageChatMemoryAdvisor`** looks up the messages for your Conversation ID, attaches them to the new prompt, sends everything to the model, and then stores the response back under the same ID.

# 3. Why Conversation ID Matters

- **Maintains context:** Lets the model see previous turns and respond naturally to follow-ups like "and what about the second one?"
- **Separates users:** Different users get different Conversation IDs, so their chats never mix.
- **Supports multiple sessions:** A single user can have several parallel chats by using a unique Conversation ID per chat thread.
- **Enables persistence:** Combined with a database-backed repository, the Conversation ID becomes the key for resuming chats across sessions.
- **Required in newer Spring AI versions:** From Spring AI 1.0.0-RC1 onwards, the framework forces you to pass it explicitly, no defaults.

# 4. The Error Faced in Class

During the live session, the following exception appeared in the console:

```
java.lang.IllegalArgumentException: conversationId cannot be null
    at org.springframework.util.Assert.notNull(Assert.java:181)
    at org.springframework.ai.chat.client.advisor.api
        .BaseChatMemoryAdvisor.getConversationId(BaseChatMemoryAdvisor.java:39)
    at org.springframework.ai.chat.client.advisor
        .MessageChatMemoryAdvisor.before(MessageChatMemoryAdvisor.java:75)
```

### Why this happened

Starting from Spring AI **1.0.0-RC1**, the Conversation ID is **no longer optional**. The framework removed the default fallback (`ChatMemory.DEFAULT_CONVERSATION_ID`) that older versions used silently behind the scenes.

Now, every call through `MessageChatMemoryAdvisor` or `VectorStoreChatMemoryAdvisor` must explicitly supply a Conversation ID. If you forget, the advisor throws `IllegalArgumentException` immediately, even before reaching the model.

> **Key Point**
> The course is using Spring AI 1.1.8, which enforces this strict rule. The first version of the code did not pass a Conversation ID, which is exactly why the exception was thrown.

## 5. The Fix Used in Class

Here is the exact controller code we ended up with after fixing the issue:

```
@RestController
public class Telusko {

    private final ChatClient chatClient;

    @Autowired
    private DateTimeTool dateTimeTool;

    @Autowired
    private NewsTool newsTool;

    ChatMemory chatMemory = MessageWindowChatMemory.builder().build();
    String conversationId = UUID.randomUUID().toString();

    public Telusko(ChatClient.Builder builder) {
        this.chatClient = builder
                .defaultAdvisors(
                    MessageChatMemoryAdvisor.builder(chatMemory).build()
                )
                .defaultAdvisors(a ->
                    a.param(ChatMemory.CONVERSATION_ID, conversationId)
                )
                .build();
    }
}
```

### What this code does

- **Creates a ChatMemory:** `MessageWindowChatMemory.builder().build()` gives us an in-memory sliding window of recent messages.
- **Generates a Conversation ID:** `UUID.randomUUID().toString()` produces a unique string for this controller instance.
- **Registers the advisor:** `MessageChatMemoryAdvisor.builder(chatMemory).build()` plugs chat memory into the ChatClient pipeline.
- **Supplies the ID:** `a.param(ChatMemory.CONVERSATION_ID, conversationId)` tells the advisor which conversation this request belongs to.
-

**Important Note**

The current code uses a single Conversation ID at the controller level. This means all users hitting this endpoint share the same conversation, which is fine for demo purposes but not suitable for production. The proper pattern is shown in the next section.

## 6. Production-Ready Pattern

In a real application, each user (or session) gets their own Conversation ID. The ID should come from your authentication layer, session, or be passed in the request.

```
@RestController
@RequestMapping("/api/chat")
public class ChatController {

    private final ChatClient chatClient;
    private final ChatMemory chatMemory;

    public ChatController(ChatClient.Builder builder, ChatMemory chatMemory) {
        this.chatMemory = chatMemory;
        this.chatClient = builder
                .defaultAdvisors(
                    MessageChatMemoryAdvisor.builder(chatMemory).build()
                )
                .build();
    }

    @PostMapping("/message")
    public String sendMessage(
            @RequestHeader("userId") String userId,
            @RequestBody String message) {

        // Each user has their own conversation
        String conversationId = "user-" + userId;

        return chatClient.prompt()
                .user(message)
                .advisors(a -> a.param(ChatMemory.CONVERSATION_ID,
conversationId))
                .call()
                .content();
    }
}
```

This pattern keeps every user's chat fully isolated. User A's history never leaks into User B's conversation.

# 7. Spring AI Version Differences

The Conversation ID API has gone through significant changes. Here is what shifted across recent versions:

| Version | Conversation ID | Builder Method | Behavior |
|---|---|---|---|
| Before 1.0.0-RC1 | Optional, fell back to default | `.conversationId()` available | Silent default, no error |
| 1.0.0-RC1 onwards | **Required, must be passed** | `.conversationId()` removed | **Throws IllegalArgumentException if null** |
| 1.1.x (current) | Required | Param-based only | Strict enforcement |
| 2.0.0 and beyond | Required | Param-based, RetrievalAugmentationAdvisor introduced | Same enforcement, expanded RAG patterns |

### Old code that no longer works

```
// Before 1.0.0-RC1 - this used to work
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultAdvisors(
        MessageChatMemoryAdvisor.builder(chatMemory)
            .conversationId("my-session")   // REMOVED in newer versions
            .build()
    )
    .build();

chatClient.prompt()
    .user("Hello!")
    .call()
    .content();
```

### Current code (Spring AI 1.1.8 and later)

```
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
    .build();

chatClient.prompt()
    .user("Hello!")
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, "my-session"))
```

```
    .call()
    .content();
```

**Another important change**

The constant CHAT_MEMORY_CONVERSATION_ID_KEY has been renamed to CONVERSATION_ID and moved from AbstractChatMemoryAdvisor to the ChatMemory interface. Always import from org.springframework.ai.chat.memory.ChatMemory.

## 8. Where to Store the Conversation ID

Once a Conversation ID is generated, you need to keep it consistent across multiple requests from the same user. Different storage options suit different needs:

| Storage | When to use | Persistence |
|---|---|---|
| HTTP Session | Short-lived chats tied to a single browser session | Session lifetime only |
| JWT Claim | Stateless APIs where token already carries user context | Token lifetime |
| Redis / Cache | Multi-instance apps needing fast lookups | Configurable TTL |
| Database | Long-term history, audit, analytics, resume chats | Permanent |
| Request Header | Client-side managed IDs (mobile apps, SPAs) | Per request |

## 9. Key Points About Conversation ID

Twelve points to remember as a first-time learner:

1. A Conversation ID is just a string. UUID, username, session ID, or any unique value works.
2. It is required in Spring AI 1.0.0-RC1 onwards. No default, no fallback.
3. If you do not pass it, you get IllegalArgumentException: conversationId cannot be null.
4. Generate it ONCE per user or chat thread, then REUSE it for all follow-up messages.
5. Generating a new ID on every API call means the model loses memory every time.
6. The same ID lets MessageChatMemoryAdvisor fetch previous messages and inject them into the new prompt.
7. Each user should have a unique Conversation ID. Sharing one ID across users will mix their chats.
8. The Conversation ID never goes to the LLM as data. It is only used internally by Spring AI to look up history.

9. Store the ID against the user in session, cache, or DB so it survives across requests.
10. Use MessageWindowChatMemory to limit how many past messages are loaded (controls token cost).
11. For production, switch from InMemoryChatMemoryRepository to JDBC, Redis, Mongo, or Cassandra repositories.
12. The ID is what makes the difference between a stateless chatbot and a stateful conversational agent.

# 10. Frequently Asked Questions

### Q1. Do we need to store the Conversation ID somewhere?

Yes. The ID must be reused for every follow-up message from the same user. Store it in session, cache, or database depending on how long the chat should persist.

### Q2. Does the Conversation ID get sent to the LLM?

No. The ID is used only by Spring AI internally to look up and store messages. The model only sees the actual chat messages.

### Q3. How does Spring AI stitch my follow-up prompt with the previous one?

When you supply a Conversation ID, MessageChatMemoryAdvisor fetches all stored messages for that ID, prepends them to the new prompt, and sends the combined history to the model.

### Q4. What happens if multiple users hit the API at the same time?

Each user should be assigned a unique Conversation ID by your backend. The ChatMemory repository is thread-safe (uses ConcurrentHashMap by default), so concurrent users are handled cleanly.

### Q5. Should I generate a new Conversation ID on every API call?

No. A new ID means the model treats every message as a brand new conversation with no memory. Generate it once per user or chat thread and reuse it.

### Q6. Can I use the username as the Conversation ID?

Yes, but only if each user has a single ongoing conversation. For multiple parallel chats per user, combine username with a session or chat ID, for example user-john-session-123.

### Q7. What is the difference between MessageWindowChatMemory and a full history?

MessageWindowChatMemory keeps only the last N messages (default sliding window) to control token usage. Full history would send everything ever said, which can blow up costs and hit context window limits.

### Q8. Why did the IllegalArgumentException happen in class?

The first version of the code did not pass a Conversation ID. From Spring AI 1.0.0-RC1 onwards, this is mandatory. Adding a.param(ChatMemory.CONVERSATION_ID, conversationId) fixed the issue.

### Q9. Does the Conversation ID work with database-backed memory like JDBC or Mongo?

Yes. The ID is the key in any ChatMemoryRepository implementation. Switching from InMemoryChatMemoryRepository to JdbcChatMemoryRepository or MongoChatMemoryRepository requires no change to the controller logic.

### Q10. How long should the conversation history live?

Depends on the use case. For one-time Q and A, session storage is enough. For ongoing user assistants, persist in a database with the user's ID as the key. MessageWindowChatMemory.maxMessages() controls how many messages are loaded into context.

### Q11. Can two users share the same Conversation ID?

Technically yes, but they will see each other's messages. Always assign unique IDs per user unless you intentionally want a shared chat room.

### Q12. What changed in Spring AI 2.0 around Conversation ID?

The strict requirement of supplying Conversation ID stays the same. Spring AI 2.0 adds RetrievalAugmentationAdvisor and new RAG pipeline patterns. The ChatMemory API itself is stable across 1.1.x and 2.0.

## 11. Quick Checklist Before You Run

- Created a ChatMemory bean (MessageWindowChatMemory or similar).
- Registered MessageChatMemoryAdvisor as a default advisor on ChatClient.
- Generated or retrieved a Conversation ID for the current user.
- Passed the ID using a.param(ChatMemory.CONVERSATION_ID, conversationId) on every prompt call.
- Reusing the same ID across follow-up requests for the same conversation.

- Using import org.springframework.ai.chat.memory.ChatMemory for the constant.
- For production: storing the ID per user, not as a single instance variable.

## 12. Official References

- **Spring AI Chat Memory Docs:** https://docs.spring.io/spring-ai/reference/api/chat-memory.html
- **Spring AI Advisors API:** https://docs.spring.io/spring-ai/reference/api/advisors.html
- **Spring AI Upgrade Notes:** https://docs.spring.io/spring-ai/reference/upgrade-notes.html
- **ChatClient API Reference:** https://docs.spring.io/spring-ai/reference/api/chatclient.html